패션 에이전트 아키텍처

앱은 HTTPS로 API Gateway 하나만 부릅니다. 백엔드는 Java · Spring Boot, agent가 Gemini로 할 일을 골라 기능별 MCP 서버를 부릅니다. 사진 분석은 우리가 학습한 모델을 서버에서 직접 돌려 호출 비용을 줄이고, 옷장 추천은 RAG로 합니다. 각 상자 아래 칩이 추천 기술 스택입니다.



일반 호출 (HTTPS · JDBC) · MCP 호출 · 노란 상자 = 데모에 꼭 필요한 경로 · 비밀 값(Gemini 키, DB 비밀번호, 문자 키)은 서버 환경 변수에만, 앱·레포에는 넣지 않음

FASHION AGENT · 요청 흐름

중심 기능: 옷 추천 (말로 추천 → 카메라로 자동 확인)

사용자는 사진을 찍지 않습니다. 먼저 말로 추천을 듣고, 자기 방식대로 꺼낸 옷을 카메라에 보여 주면 알아서 판단해 줍니다. 오늘 면접이 있는 경우입니다.

1
"오늘 뭐 입지?"

agent가 오늘 날씨, 오늘 일정과 장소 종류, (선택) 옷장 후보를 동시에 모읍니다.

`mcp-weather · mcp-schedule · mcp-closet(RAG)`

2
말로 먼저 추천

Gemini가 추천합니다. "체감 9도에 면접이 있어요. 어두운 슬랙스에 셔츠, 겉옷을 추천드려요." 앱이 카메라를 켭니다.

`agent · Gemini 추천 모드`

3
폰이 실시간으로 봄

ML Kit로 매 순간 옷이 잘 보이는지 확인하고 "조금 왼쪽으로"를 바로 말합니다. 서버에 가지 않아 비용이 없습니다.

`application · ML Kit 물체 인식`

4
멈추면 한 장 자동 전송

옷이 잘 들어와 1초 멈추면 한 장만 보냅니다. 버튼을 누르지 않고, 사진은 저장하지 않습니다.

`gateway → agent → mcp-clothes-state`

5
Gemini가 판단·비교

1단계 추천과 앞서 본 옷까지 보고 판단합니다. 두 벌이면 "왼쪽 흰 셔츠가 더 단정해요. 겉옷도 보여 주세요."

`agent · Gemini 확인 모드`

6
"이걸로 할게"

본 옷과 입은 날을 옷장에 기록하고 마칩니다. 다음 추천부터 옷장 후보가 쌓입니다.

`mcp-closet · closet_save · closet_wear`

왼쪽·오른쪽: 옷끼리 비교는 사용자가 선 자리 기준(= 사진 기준), 옷 한 벌 안의 얼룩 위치는 입은 사람 기준. 비교할 때는 "왼쪽 흰 셔츠"처럼 색·종류를 붙여 손으로 한 번 더 확인할 수 있게 합니다.
개별 기능은 따로 부릅니다: 날씨 묻기, 일정 등록, 얼굴 확인(face-check), 가족·친구에게 확인 요청(helper). 이때는 Gemini가 부를 도구를 고릅니다.

MCP는 데이터, 판단은 Gemini

MCP 서버는 옷 상태·날씨·일정·옷장 데이터만 돌려줍니다. 사진은 우리 모델이 서버에서 직접 보고, Gemini는 사진 없이 JSON만 받아 판단과 추천을 합니다.

확실하지 않으면 단정하지 않기

얼룩 확신도 0.7 미만은 "손으로 확인해 보세요"라고만 말합니다. 옷장 추천은 RAG로 찾은 내 옷 안에서만 하고, 없는 옷을 지어내지 않습니다.

비밀 값과 사진은 밖으로 새지 않게

API 키는 서버 환경 변수에만. 사진은 저장하지 않고, helper로 보낼 때만 24시간 뒤 지워지는 곳에 둡니다.

레포별 추천 기술과 대안

구성요소	레포	추천 기술 스택	고른 이유	대안
application	`fashion-agent`	Flutter, speech_to_text, flutter_tts, camera 이미지 스트림, ML Kit 물체 인식(구도 실시간·자동 전송), Firebase Auth SDK, FCM	이미 동작 중. 기기 음성은 무료이고 VoiceOver 목소리와 같음	Google Cloud STT/TTS, 네이버 CLOVA (실기기에서 인식이 부족할 때)
API Gateway	`fashion-agent-gateway`	Spring Cloud Gateway, Redis 사용량 제한	팀 스택과 같고 라우팅·JWT 필터·사용량 제한을 설정으로 처리	Nginx, 발표 전엔 게이트웨이 없이 agent에 Spring Security
account · 회원정보	`fashion-agent-gateway`	Spring Security, Firebase Admin(ID 토큰 검증), Spring Data JPA	Apple·구글 로그인과 기기 이전용 계정 연결을 Firebase가 맡고 서버는 확인만	Spring Security OAuth2 + 자체 JWT, Keycloak
agent	`fashion-agent-agent`	Spring AI ChatClient, MCP 클라이언트, Gemini 3.5 Flash, ChatMemory	MCP 서버와 같은 프레임워크, 서버 목록만 적으면 도구가 붙음	LangChain4j, Google Gen AI Java SDK 직접 호출
MCP 공통	`mcp-*`	Spring AI MCP Server (Streamable HTTP, @Tool)	메서드에 어노테이션만 붙이면 도구가 됨. agent와 설정 방식이 같음	MCP Java SDK 직접, TypeScript MCP SDK(Workers)
weather	`mcp-weather`	Open-Meteo, RestClient, Caffeine 캐시	키 없음, 앱에서 이미 검증, 체감 온도·일교차가 한 번에	기상청 단기예보 API, OpenWeatherMap
schedule	`mcp-schedule`	카카오 로컬 API, FCM, @Scheduled	국내 장소 카테고리(예식장·장례식장)가 정확, 알림은 무료	네이버 지역 검색 + 폰 로컬 알림, Google Places + Calendar
clothes_state	`mcp-clothes-state`	얼룩·찢어짐·보풀: YOLO 파인튜닝 / 종류·격식: FashionCLIP(학습 없이) / 색: 픽셀 평균 → 색 이름 규칙 / 실행: ONNX Runtime Java / 폴백: Gemini	호출 비용 없이 서버 CPU로 1초 안팎. 위치(박스)가 나와서 "오른쪽 소매 끝" 안내 가능. 데이터가 필요한 건 결함 탐지 하나	Python FastAPI 추론 서버(MCP는 Java 유지), 앱 안 Core ML·TFLite
face_check	`mcp-face-check`	앱 ML Kit 얼굴 윤곽점 → 서버 대칭 규칙(눈썹·입술 위치, 볼·입술 색 좌우 비교) + 입 부분 치아 이물질 분류(MobileNet 파인튜닝, ONNX)	대칭은 학습 없이 계산. 학습이 필요한 건 치아 분류 하나라 데이터가 적게 듦	MediaPipe Face Landmarker(점 478개, Python 서버), 전부 앱 안 TFLite
closet (RAG)	`mcp-closet`	Spring AI VectorStore + pgvector, 임베딩 multilingual-e5-small(ONNX, 서버에서 무료), 답변 Gemini Flash. 검색 대상: 내 옷 설명 + 코디 지식 문서(드레스코드·색 조합)	가진 옷 안에서만, 어떤 옷·규칙을 근거로 했는지 말하며 추천. 같은 옷 중복 찾기도 같은 벡터로	Gemini 임베딩 API, Qdrant·Chroma 벡터 DB, LangChain4j
모델 학습	`각 mcp 레포 training/`	Python, Ultralytics YOLO, Roboflow 라벨링, Google Colab GPU, ONNX 내보내기	학습은 Python이 표준, ONNX로 내보내면 Java에서 그대로 실행	학교·개인 GPU, AWS SageMaker
helper	`mcp-helper`	문자(NHN Cloud SMS 또는 솔라피), 답장용 웹 링크, Cloudflare R2	가족은 앱 설치 없이 링크로 답함, 카카오 심사 없이 바로 가능	카카오 알림톡(채널·템플릿 심사), 같은 앱 도우미 모드 + FCM
DB	(공용)	PostgreSQL + pgvector	회원·일정 데이터와 RAG 벡터를 한 DB에	MySQL + 별도 벡터 DB, Firestore
배포	(공용)	Docker Compose, VM 한 대, Nginx/Caddy HTTPS	서비스 전부를 파일 하나로 올리고 내림	Google Cloud Run, Kubernetes(발표 후)

학습 데이터: 공개 원단 결함 데이터(AITEX 등)는 공장 원단 사진이라 펼쳐 놓은 옷과 달라서, 우리 옷 사진을 직접 찍어 라벨링해야 합니다(결함 종류마다 수백 장 목표). 지금 배포된 Gemini 옷 분석 서버는 모델이 준비될 때까지 데모용 예비로 켜 둡니다.

FASHION AGENT · 시작 순서 (제안)

한 명이 레포 하나씩, 데모 경로부터

레포 9개를 동시에 시작하지 않고, "오늘 뭐 입지?"부터 옷 확인까지 끝까지 도는 경로의 레포 3개를 먼저 한 명씩 맡습니다. 첫날 인터페이스를 같이 정해 두면 각자 따로 만들어도 마지막에 바로 붙습니다.

11/4 – 11/5

0. 약속 정하기

- 앱 ↔ 게이트웨이 API 모양
- MCP 도구 이름·입출력
- JWT에 담을 값 (uid)
- Docker Compose 뼈대, .env 목록
- 학습용 사진 찍기·라벨링 시작

11/6 – 11/12

1. 데모 경로

- 게이트웨이 + account
- agent + mcp-weather
- clothes-state 모델 1차 학습 + MCP 서버
- 앱: 1단계 추천 → 카메라 자동 전송

11/13 – 11/17

2. 기능 넓히기

- mcp-schedule (음성 일정 등록)
- mcp-closet RAG (옷 설명 임베딩·검색)
- 개별 기능: face-check, helper
- 모델 2차 학습 (틀린 사진 추가)
- 폰 자동 전송 규칙 다듬기

11/18 – 11/19

3. 리허설

- 실제 아이폰으로 처음부터 끝까지
- 예비 서버로 바꾸는 방법 확인
- 발표 시나리오 녹화 (네트워크 대비)

11/20 발표

담당 A

fashion-agent-gateway

라우팅, JWT 검증, 사용량 제한, Firebase 토큰 확인, 회원정보 저장. Docker Compose 뼈대도 같이.

담당 B

fashion-agent-agent (+ mcp-weather)

Spring AI로 Gemini 연결, MCP 클라이언트, 대화 기억, 결과를 한국어 안내 문장으로. weather는 작아서 함께.

담당 C

mcp-clothes-state

얼룩·찢어짐 데이터 라벨링과 YOLO 학습(Colab), ONNX로 내보내 Spring AI MCP에서 실행. 응답 모양은 지금 ClothesScan 그대로. 모델이 늦으면 Gemini 폴백으로 데모.

```
// 앱 → 게이트웨이 (docs/contracts/openapi.yaml)
POST /api/v1/agent/turn   Authorization: Bearer <JWT>
1단계 { "utterance": "오늘 뭐 입지?", "location": {…} }
  → { "speech": "…추천드려요", "next": "camera_outfit" }
2단계 { "sessionId", "imageKind": "outfit",
        "image": "<자동 전송 한 장>", "autoCaptured": true }
  → { "speech": "왼쪽 흰 셔츠가…", "next": "camera_outfit" }
```

```
// 1단계 말로 추천
weather_today · schedule_list · closet_search(선택)
  → Gemini 추천 모드 → { suggestions, closetPicks, speech }
// 2단계 자동 전송된 한 장마다
clothes_inspect(image) → garments(+position, defects)
  → Gemini 확인 모드 → { verdicts, comparison,
                         damage, stillMissing, speech }
```